

VIII. Syntaxe CSS et propriétés essentielles

EN UNE PHRASE

La syntaxe `sélecteur { propriété: valeur; }` + les propriétés essentielles (couleurs, typo, dimensions, fonds, bordures, ombres) couvrent 80 % de vos besoins au quotidien.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Anatomie d'une règle CSS**
- **B. Les 3 sélecteurs de base**
 - 1. Sélecteur de type
 - 2. Sélecteur de classe** (à préférer)
 - 3. Sélecteur d'identifiant
- **C. Unités CSS**
 - 1. Unités absolues
 - 2. Unités relatives (à préférer)
 - 3. Exemple
 - 4. Pourquoi pas px partout ?
- **D. Couleurs**
 - 1. Les 4 formats
 - 2. Quand utiliser quel format ?
 - 3. Format hexadécimal — mémo
 - 4. Contraste et accessibilité
- **E. Typographie**
 - Pile de polices système (performances max)
 - Google Fonts
- **F. Dimensions, fonds, bordures, ombres**
 - 1. Dimensions
 - 2. Centrer horizontalement un bloc
 - 3. Fonds
 - 4. Bordures et coins arrondis
 - 5. Ombres
 - 6. Opacité
 - 7. Curseur
- **G. Commentaires**
 - Bonnes pratiques d'organisation
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Écrire** une règle CSS valide (sélecteur + déclarations).
- **Identifier** les 3 sélecteurs de base (type, classe, id) et les différencier.
- **Utiliser** les unités CSS (`px` , `rem` , `em` , `%` , `vw` / `vh`) à bon escient.
- **Appliquer** les propriétés essentielles : couleurs (hex, RGB, HSL), typographie, dimensions, fonds, bordures, ombres.
- **Commenter** son code CSS pour le rendre lisible.

2. En pratique — situation professionnelle

Le designer vous envoie une maquette de carte produit : fond blanc, ombre douce, coins arrondis, titre gris foncé 18px, prix orange 24px gras, bouton bleu. Cette leçon vous donne toutes les propriétés nécessaires pour reproduire cette carte sans rien chercher sur Internet.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Stylisation initiale d'un composant (carte, bouton, encart)	<code>px</code> partout sur les <code>font-size</code> (accessibilité KO)
<code>rem</code> pour la typo, <code>%</code> / <code>vw</code> pour les largeurs fluides	Couleur en mot-clé (<code>red</code>) en prod : préférer hex de la charte
Hexadécimal pour la charte, <code>rgba()</code> pour la transparence	<code>opacity</code> quand seul le fond doit être translucide

A. Anatomie d'une règle CSS

```
h1 {                               /* ← sélecteur */
  color: red;                      /* ← propriété: valeur; (une déclaration) */
  font-size: 24px;                /* ← autre déclaration */
}
```

Élément	Rôle	Exemple
Sélecteur	Qui on cible	h1
Déclaration	Paire propriété + valeur	color: red;
Propriété	Ce qu'on modifie	color
Valeur	La nouvelle valeur	red

À ne jamais oublier :

- Le **point-virgule** ; après chaque déclaration.
- Les **accolades** { } autour du bloc.
- Le **deux-points** : entre propriété et valeur (pas =).

B. Les 3 sélecteurs de base

1. Sélecteur de type

Cible **tous les éléments** d'un type HTML.

```
p {  
  color: blue;  
}
```

→ Tous les `<p>` deviennent bleus. Utile pour les styles globaux (typo du body, style des titres).

2. Sélecteur de classe** (à préférer)

Cible tous les éléments ayant l'attribut `class` correspondant. Préfixé par **un point** `.`.

```
<button class="btn">OK</button>  
<button class="btn">Annuler</button>
```

```
.btn {  
  background: #eee;  
  padding: 8px 16px;  
}
```

Quand utiliser : le plus courant, à préférer par défaut. Styles réutilisables.

3. Sélecteur d'identifiant

Cible l'élément ayant l'attribut `id` correspondant (unique par page). Préfixé par **#** `#`.

```
#main-header {  
  background: navy;  
}
```

Quand utiliser : rarement. Réserver l'`id` pour les ancres, le JavaScript (`getElementById`), les labels de formulaire.

◆ Règle du pouce

90 % classes, 9 % sélecteurs de type, 1 % IDs.

Les sélecteurs avancés (combinés, attributs, pseudo-classes, pseudo-éléments) sont détaillés au chapitre IX.

C. Unités CSS

1. Unités absolues

- **px** (pixel) — taille fixe. OK pour les bordures et petits détails, **à éviter** pour les tailles de police.

2. Unités relatives (à préférer)

Unité	Calcul	Usage
em	Relatif au parent	Tailles internes d'un composant
rem	Relatif à la racine (<code><html></code>)	Tailles globales, marges, espacements
%	Pourcentage du parent	Largeurs fluides
vw / vh	% largeur/hauteur du viewport	Hero sections plein écran

3. Exemple

Avec `<html>` à `font-size: 16px` :

```
h1      { font-size: 2rem; }      /* = 32px */
.article { padding: 1.5rem; }     /* = 24px */
.hero    { height: 100vh; padding: 0 5vw; }
```

4. Pourquoi pas **px** partout ?

Parce que les utilisateurs peuvent **agrandir la taille de police** dans les options de leur navigateur (accessibilité malvoyants). Avec **rem**, votre site s'adapte automatiquement. Avec **px**, il reste figé.

◆ Règle simple

rem pour typo et espacements globaux, **% / vw** pour largeurs fluides, **px** uniquement pour les bordures fines (`border: 1px solid`).

⚠ Piège de l'accumulation `em`

Trois niveaux imbriqués de `font-size: 1.2em` donnent **+73 %** de taille (pas +20 %). Préférer **rem** qui est absolu.

D. Couleurs

1. Les 4 formats

```
.elem {
  color: red;                /* 1. Mot-clé (~150 dispos) */
  color: #FF0000;           /* 2. Hexadécimal */
  color: rgb(255, 0, 0);    /* 3. RGB */
  color: rgba(255, 0, 0, 0.5); /* 4. RGBA (transparence 0→1) */
  color: hsl(0, 100%, 50%); /* 5. HSL */
}
```

2. Quand utiliser quel format ?

Format	Cas d'usage
Mot-clé	Prototypage rapide, debug
Hexadécimal <code>#RRGGBB</code>	Le plus courant, charte graphique
RGBA	Quand il faut de la transparence
HSL	Variations d'une couleur (+ sombre, + saturée)

3. Format hexadécimal — mémo

- `#RRGGBB` : rouge/vert/bleu sur 2 caractères chacun (`00` - `FF`).
- `#RGB` : raccourci, ex `#F0A` = `#FF00AA` .
- `#RRGGBBAA` : avec canal alpha.

`#000000` = noir, `#FFFFFF` = blanc, `#808080` = gris.

4. Contraste et accessibilité

⚠ Gris clair sur fond blanc = illisible

Ratio de contraste minimum **4.5:1** (norme WCAG AA, texte courant). Vérifier avec :

- Chrome DevTools → inspecter un élément → onglet *Accessibility* → Contrast.
- WebAIM Contrast Checker**

E. Typographie

```
body {  
  font-family: "Segoe UI", Roboto, Arial, sans-serif;  
  font-size: 1rem;  
  font-weight: 400;  
  line-height: 1.5;  
  color: #222;  
}  
  
h1 {  
  font-family: "Georgia", serif;  
  font-size: 2.5rem;  
  font-weight: 700;  
  letter-spacing: -0.02em;  
  text-transform: uppercase;  
}
```

Propriété	Valeurs
font-family	Liste de fallback séparée par virgules, dernière valeur générique (<code>sans-serif</code>)
font-size	<code>1rem</code> , <code>1.25em</code> (éviter <code>px</code>)
font-weight	<code>normal</code> (400), <code>bold</code> (700), ou 100-900
line-height	<code>1.5</code> (multiplicateur, sans unité) — à privilégier
text-align	<code>left</code> , <code>right</code> , <code>center</code> , <code>justify</code>
text-transform	<code>uppercase</code> , <code>lowercase</code> , <code>capitalize</code>
text-decoration	<code>none</code> , <code>underline</code> , <code>line-through</code>

Pile de polices système (performances max)

```
body {  
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,  
               Helvetica, Arial, sans-serif;  
}
```

Utilise la police native du système → pas de téléchargement, rendu natif.

Google Fonts

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap"
      rel="stylesheet">
```

```
body { font-family: "Inter", sans-serif; }
```

♦ `&display=swap` dans l'URL

Sans ça, le texte reste **invisible** tant que la police n'est pas chargée (FOIT). Avec `swap`, le texte apparaît dans une fallback puis bascule.

F. Dimensions, fonds, bordures, ombres

1. Dimensions

```
.carte {  
  width: 300px;  
  max-width: 100%; /* ne dépasse jamais le parent */  
  min-width: 200px;  
  
  height: auto;  
  min-height: 100px;  
}
```

♦ Le duo magique `width: 400px; max-width: 100%;``

Largeur max 400px sur grand écran, mais jamais plus large que le parent sur mobile.
Classique sur cartes et images responsives.

2. Centrer horizontalement un bloc

```
.centre {  
  margin: 0 auto; /* 0 haut/bas, auto gauche/droite → centré */  
}
```

Requiert une `max-width` ou `width` définie.

3. Fonds

```
.section {  
  background-color: #f8f9fa;  
  
  /* Image */  
  background-image: url("hero.jpg");  
  background-size: cover;          /* couvre tout */  
  background-position: center;  
  background-repeat: no-repeat;  
  
  /* Dégradé */  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  
  /* Raccourci tout-en-un */  
  background: #eee url("bg.jpg") no-repeat center / cover;  
}
```

4. Bordures et coins arrondis

```
.boite {  
  border: 2px solid #333;          /* épaisseur style couleur */  
  border-top: 1px solid #ccc;      /* un seul côté */  
  border-radius: 8px;              /* tous les coins */  
  border-radius: 50%;              /* rond parfait (si width = height) */  
}
```

5. Ombres

```
.carte {  
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);  
  /*      X  Y  flou couleur */  
}  
  
/* Ombres empilées */  
.carte-elevee {  
  box-shadow:  
    0 4px 8px rgba(0, 0, 0, 0.1),  
    0 1px 2px rgba(0, 0, 0, 0.15);  
}  
  
/* Ombre interne */  
.bouton-presse {  
  box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.2);  
}
```

6. Opacité

```
.overlay { opacity: 0.5; } /* 0 invisible → 1 opaque */
```

⚠ `opacity` affecte TOUT l'élément et ses enfants

Pour rendre **seulement le fond** transparent en gardant le texte opaque, utiliser `rgba()` sur `background-color`, pas `opacity`.

7. Curseur

```
button { cursor: pointer; }  
.disabled { cursor: not-allowed; }
```

G. Commentaires

CSS n'a **pas** de `//` (comme en JS). Uniquement `/* ... */` (multiligne autorisée).

```
/* Commentaire une ligne */

/*
  Commentaire
  multi-lignes
*/

.btn {
  background: blue; /* couleur de la marque */
}
```

Bonnes pratiques d'organisation

```
/* =====
1. Reset et bases
===== */

*, *::before, *::after {
  box-sizing: border-box;
}

/* =====
2. Composants
===== */

/* --- Boutons --- */
.btn { /* ... */ }
.btn-primary { /* ... */ }
```

Conclusion

1. À retenir

- Règle : `sélecteur { propriété: valeur; }`, **toujours le ;** à la fin.
- 3 sélecteurs de base : **type** (`h1`), **classe** (`.nom`), **id** (`#nom`). **Classes à 90 %.**
- Unités : `rem` pour textes/espacements, `%` / `vw` pour fluides, `px` pour détails fins. **Jamais de `px` pour une `font-size`.**
- Couleurs : **hexadécimal** par défaut, **RGBA** pour transparence.
- `box-shadow: 0 2px 4px rgba(0,0,0,0.1)` pour une ombre douce universelle.
- `margin: 0 auto` pour centrer horizontalement un bloc de largeur connue.
- Préférer `rgba()` sur `background-color` plutôt que `opacity` (qui affecte les enfants).

2. Pour aller plus loin

- **MDN — Référence des propriétés CSS**
- **CSS-Tricks — Almanac**
- **Google Fonts**
- **WebAIM — Contrast Checker**